

RESPONSIVIDADE E ADAPTABILIDADE NA WEB

Aula 1

FUNDAMENTOS CSS E APLICAÇÕES PARA O AMBIENTE WEB

Fundamentos CSS e aplicações para o ambiente Web

Caro estudante,

É com entusiasmo que convidamos você a participar desta videoaula, preparada para proporcionar uma imersão completa nos fundamentos do *Cascading Style Sheets* (CSS). Os fundamentos do CSS e sua aplicação são elementos essenciais no desenvolvimento web, e esta aula tem como objetivo orientar você pelos conceitos fundamentais, apresentando diferentes técnicas e práticas que são fundamentais para a construção de layouts e estilos em páginas da web.

Além disso, abordaremos tópicos relacionados à introdução ao CSS, como conceitos básicos, seletores e propriedades, e sua aplicação em exemplos práticos do mundo real, essenciais para uma compreensão sólida.

Não perca a oportunidade de aprimorar suas habilidades e ampliar seu conhecimento sobre os fundamentos do CSS. Junte-se a nós nessa jornada de aprendizado e fortaleça suas habilidades na área. Contamos com a sua participação!

Ponto de Partida

Olá, estudante. Desejamos boas-vindas a você para esta aula, e esperamos que esse conteúdo tenha valor agregado na construção do seu conhecimento.

A partir de agora, iniciaremos juntos a compreensão dos princípios do CSS, uma base indispensável para a construção de layouts e estilos em páginas da web. Ao longo desta aula, vamos começar a compreender os desafios da estilização eficiente de elementos HTML, proporcionando uma experiência visual atraente e funcional para os usuários.

Atualmente, o conhecimento em CSS é essencial para o desenvolvimento de sistemas e aplicativos na sociedade digital.

Desenvolver habilidades em CSS fortalece o raciocínio lógico e amplia suas oportunidades de atuação profissional em diversas áreas do conhecimento.

No decorrer da disciplina serão abordados conteúdos que introduzirão você ao mundo da estilização do CSS. Vamos conhecer as definições e conceitos, bem como o módulo CSS3 e suas propriedades, além de aprender a vinculação de folhas de estilos a documentos HTMLs.

Ao final desta aula, você terá uma compreensão teórica sólida sobre os fundamentos do CSS e estará apto a aplicá-los em situações práticas do mundo real, criando layouts responsivos e atraentes para websites e aplicativos.

Diante disso, será apresentado o desenvolvimento de uma página básica utilizando HTML e CSS. A página terá o objetivo de fazer uma apresentação pessoal.

Convidamos você a iniciar a construção de uma base sólida para sua jornada no universo da programação web a partir dos conhecimentos em CSS. Preparado para essa imersão?

Vamos Começar!

O objetivo desta aula é apresentar a introdução à conceitos fundamentais do *Cascading Style Sheets* (CSS), suas principais características, além de iniciar você, estudante, neste enorme cenário do design de uma página web.

O CSS, que significa *Cascading Style Sheets* (folhas de estilo em cascata), é uma linguagem de estilo utilizada para definir a apresentação de documentos escritos em uma linguagem de marcação, como HTML.

Conceitos e introdução ao CSS

Esse sistema de estilização de páginas web, ou simplesmente "CSS", como ficou conhecido, foi desenvolvido inicialmente por Håkon Wium Lie e Bert Bos em 1996, como uma forma de separar a estrutura do conteúdo do documento de sua apresentação visual.

O surgimento do CSS foi uma resposta à necessidade de controlar o layout e a aparência dos sites de uma maneira mais eficiente, em comparação com as técnicas de formatação *inline* e tabelas que eram comuns na época. A separação de estilo do conteúdo trouxe uma série de benefícios, incluindo uma maior flexibilidade, manutenibilidade e acessibilidade dos sites.

A primeira versão do CSS, conhecida como CSS1, foi padronizada pelo *World Wide Web Consortium* (W3C) em 1996. Ela oferecia recursos básicos de formatação, como controle de cores, fontes e layout de página. Com o tempo, o CSS passou por várias atualizações e evoluções para acompanhar as demandas crescentes da web moderna.

O CSS2 foi lançado em 1998, introduzindo recursos avançados, como posicionamento de elementos, efeitos visuais e suporte para mídia impressa. No entanto, foi o CSS3, dividido em vários módulos, que trouxe uma ampla gama de novos recursos, incluindo animações, gradientes, sombras, efeitos de transformação e muito mais.

Atualmente, o CSS3 é amplamente utilizado e continua a ser desenvolvido e aprimorado pela comunidade de desenvolvedores e pelo W3C. Além disso, novas tecnologias, como pré-processadores CSS (por exemplo, SASS e LESS) e frameworks CSS (como Bootstrap e Foundation), tornaram o desenvolvimento e a manutenção de estilos web ainda mais eficientes e poderosos.

Uma das características fundamentais do CSS é a sua capacidade de aplicar estilos de forma hierárquica e cascata. Isso significa que os estilos podem ser definidos em diferentes níveis e, em caso de conflito, as regras são aplicadas de acordo com uma ordem de precedência definida. Por exemplo, estilos definidos diretamente em

um elemento HTML têm precedência sobre estilos definidos em uma folha de estilo externa. Observe a Figura 1, com um exemplo dessa hierarquia.

```
1  /* Estilo aplicado a todos os parágrafos */
2  v p {
3      color: blue;
4  }
5
6  /* Estilo específico para parágrafos dentro de um div com a classe "container" */
7  v .container p {
8      color: red;
9  }
10
11 v /* Estilo ainda mais específico para um parágrafo com a classe
12 |  "destaque" dentro do div com a classe "container" */
13 v .container .destaque {
14     font-weight: bold;
15 }
16
```

Figura 1 | Conceito de hierarquia e cascata.

Na linha 1 da Figura 1, observe que o a classe "p" é um estilo que será aplicado para todos os parágrafos. Já na linha 7, o ".container p" é específico para parágrafos que se encontram dentro de uma "div" com a classe "container", e por fim, na linha 12, o ".container.destaque" é específico para um parágrafo com a classe "destaque" dentro do "div" com a classe "container".

Para compreender um pouco melhor como o CSS vai estilizar ou formatar os conteúdos HTML em uma página web, devemos entender também o seu funcionamento.

O CSS trabalha em conjunto com o HTML, pois dentro de um arquivo HTML podemos inserir comandos em estilo CSS. Procurando dar uma visão prática, vamos apresentar algumas características de funcionamento do CSS.

Em primeiro lugar é preciso dizer ao documento HTML que ele precisa aceitar comandos em CSS. Este processo é realizado por meio da tag: <link rel="stylesheet">.

Dentro do código fonte HTML é simples fazer esta referência, e ela pode ser feita de duas maneiras diferentes. Uma é fazer referência ao código CSS dentro do próprio arquivo e a outra maneira é fazer

referência a um arquivo externo. O comportamento em relação à execução dos comandos será o mesmo, mas a organização dos comandos será feita de forma diferente. Enfim, são possibilidades de trabalho com o conceito do CSS.

```
1  <!DOCTYPE html>
2  <html lang="pt-br">
3
4  <head>
5      <meta charset="UTF-8">
6      <meta name="viewport" content="width=device-width, initial-scale=1.0">
7      <title>Nomes Centralizados</title>
8      <link rel="stylesheet" href="style.css">
9  </head>
10
11 <body>
12     <div class="minhdiv">
13
14         <div e class="bloco"> França </div>
15         <div e class="bloco"> Brasil </div>
16         <div e class="bloco"> Itália </div>
17         <div e class="bloco"> EUA </div>
18         <div e class="bloco"> Alemanha </div>
19         <div e class="bloco"> Turquia </div>
20
21     </div>
22 </body>
23
24 </html>
```

Figura 2 | CSS em arquivo externo.

A Figura 2 apresenta na linha 8 o link para outro arquivo com o nome de "style.css", onde existem as configurações de design de todo o documento index.html exibido na figura. Observe a seguir o conteúdo do arquivo "style.css", exibido na Figura 3.

```
1  ∨ *{
2      margin:0;
3  }
4  ∨ .minhdiv {
5      width: 100%;
6      height: 600px;
7      background-color: □black;
8      display: grid;
9      grid-template-columns: 1fr;
10     column-gap: 10px;
11     row-gap: 10px;
12     text-transform: uppercase;
13     position: absolute;
14     margin: 15px;
15
16 }
17 ∨ .bloco {
18     width: 100%;
19     height: 35px;
20     background-color: ■white;
21     text-align: center;
22 }
23 ∨ @media(min-width: 480px) {
24     ∨ .minhdiv{
25         grid-template-columns: repeat(2, 1fr);
26     }
27 }
28 ∨ @media(min-width: 768px) {
29
30     ∨ .minhdiv{
31         grid-template-columns: repeat(3, 1fr);
32     }
33
34 }
35 ∨ @media(min-width: 1280px) {
36
37     ∨ .minhdiv{
38         grid-template-columns: repeat(6, 1fr);
39     }
40 }
```

Figura 3 | Código do arquivo "style.css".

A Figura 3 apresenta parte da configuração de CSS existente que será responsável pela formatação do conteúdo da página index.html no navegador web.

Vamos apresentar esta página no navegador web e nos certificar de que realmente os dois arquivos estão conectados, pois as configurações codificadas no arquivo CSS estão realmente interferindo na exibição do arquivo index.html.

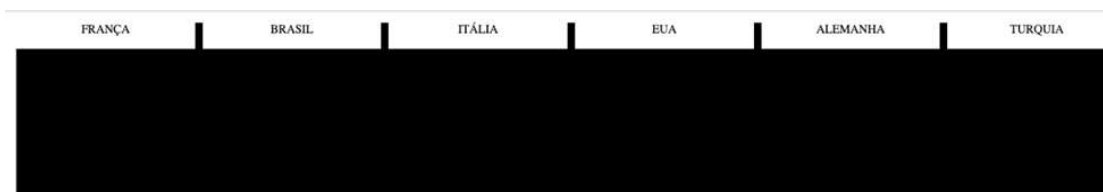


Figura 4 | Página index.html exibida no navegador.

Incrível, não é mesmo? O código CSS presente no arquivo "style.css" exibiu o conteúdo como um menu de funcionalidades – os seis países mais visitados do mundo todo.

Módulo CSS3 e status dos módulos

Bem, como você já tem uma ideia do que é o CSS, suas características e função dentro da programação de aplicações web, vamos abordar os seus módulos.

Os módulos CSS3 são conjuntos de funcionalidades e recursos adicionados ao CSS para fornecer maior flexibilidade e capacidade de design nas páginas da web. Cada módulo geralmente se concentra em uma área específica do design, como layout, tipografia e efeitos visuais, entre outros. O CSS3 é uma versão do CSS que inclui vários desses módulos, sendo uma atualização significativa em relação às versões anteriores.

Somente para informar, duas das atualizações mais significativas que o CSS3 trouxe em relação ao CSS são apresentadas a seguir:

- **Flexibilidade no layout com Flexbox e Grid Layout:** o CSS3 introduziu os módulos Flexbox e Grid Layout, que revolucionaram a maneira como os layouts são criados em páginas da web.

Esses módulos oferecem uma abordagem mais flexível e poderosa para o posicionamento e o dimensionamento de elementos em relação ao modelo tradicional de layout baseado em caixas. Com Flexbox e Grid Layout, os desenvolvedores têm mais controle sobre a organização e o alinhamento dos elementos em uma página, permitindo layouts responsivos e complexos com menos código e maior clareza.

- **Adição de recursos visuais avançados:** o CSS3 introduziu uma série de recursos visuais avançados que permitem aos desenvolvedores criar designs mais ricos e interativos. Isso inclui transformações 2D e 3D, transições e animações, filtros de imagem, sombras e gradientes. Esses recursos proporcionam uma experiência visual mais dinâmica e atraente para os usuários, permitindo que os desenvolvedores criem interfaces mais sofisticadas e modernas sem depender de recursos externos, como imagens ou scripts JavaScript.

Por sua vez, o "status dos módulos" refere-se ao estágio de desenvolvimento de cada módulo CSS3. Os módulos podem estar em diferentes estágios, indicando seu nível de maturidade e aceitação pela comunidade de desenvolvimento web. Os principais estágios são:

1. **Rascunho (Working Draft):** nesse estágio, o módulo está em desenvolvimento inicial. Ainda não é considerado uma especificação completa e pode passar por revisões significativas.
2. **Candidato a recomendação (Candidate Recommendation):** nesse estágio, o módulo é considerado estável o suficiente para ser implementado em navegadores, mas ainda está sujeito a testes e feedback da comunidade.
3. **Recomendação (Recommendation):** quando um módulo atinge esse estágio, ele é considerado uma especificação completa e estável, amplamente aceita e adotada pela comunidade de desenvolvimento web.

A seguir, vamos apresentar alguns exemplos de módulos CSS3 e seus status:

- **Flexbox:** um módulo que oferece um novo modelo de layout bidimensional mais eficiente, permitindo o alinhamento e a distribuição de elementos em um contêiner de forma mais flexível. O Flexbox é um exemplo de um módulo que alcançou o status de Recomendação. A seguir, a Figura 5 apresenta um exemplo de como usar o Flexbox com CSS3 para criar um layout de lista de itens flexíveis:

```
7 <style>
8   .container {
9       display: flex;
10      flex-direction: row;
11      justify-content: space-around;
12      align-items: center;
13      background-color: #f0f0f0;
14      padding: 20px;
15  }
16  .item {
17      background-color: #3498db;
18      color: #fff;
19      padding: 10px;
20      margin: 5px;
21      text-align: center;
22      flex: 1;
23  }
24 </style>
```

Figura 5 | Exemplo de Flexbox.

Neste exemplo, usamos o Flexbox para criar um contêiner flexível (container) e três itens flexíveis (item) dentro dele.

Dando sequência, outros módulos CSS3 e seus status são:

- **Grid Layout:** layout que introduz um sistema de grade bidimensional para organizar elementos em uma página da web. O Grid Layout também é uma Recomendação.
- **Transformações 2D e 3D:** um conjunto de propriedades que permite transformar elementos, como rotação, escala, inclinação e translação, em dois ou três planos. Esse módulo já atingiu o status de Recomendação.
- **Transições e animações:** propriedades que permitem criar transições suaves entre os estados de um elemento e animações personalizadas. Esse módulo também é uma Recomendação.

O módulo CSS3 e o status dos módulos representam uma parte fundamental do desenvolvimento web moderno. O CSS3 trouxe uma série de recursos avançados e melhorias significativas em relação às versões anteriores do CSS, permitindo aos desenvolvedores criar layouts mais complexos, estilos mais ricos e interações mais dinâmicas.

O status dos módulos, como "working draft" ou "recommendation", indica o estágio de desenvolvimento de uma especificação CSS, fornecendo aos desenvolvedores insights sobre sua estabilidade e adoção pelos navegadores. É importante acompanhar de perto o status dos módulos CSS para garantir a conformidade com as últimas práticas e padrões da web.

Portanto, compreender o módulo CSS3 e o status dos módulos é essencial para criar experiências web modernas, responsivas e compatíveis com os padrões, permitindo que os desenvolvedores aproveitem ao máximo as capacidades do CSS para estilizar e formatar conteúdo na web.

Siga em Frente...

Vinculando folhas de estilo a documentos

Vincular folhas de estilo a documentos HTML é um processo fundamental no desenvolvimento web, e você associa arquivos CSS externos a documentos HTML para controlar a aparência e o layout do conteúdo. Esse processo permite que você separe o conteúdo estrutural do conteúdo de apresentação, facilitando a manutenção e a escalabilidade do seu código.

Processo de vinculação:

- **Link Externo (External Link):** o método mais comum de vincular folhas de estilo a documentos HTML é usando a tag `<link>`. Você

especifica o arquivo CSS desejado no atributo "href", por exemplo:

```
<link rel="stylesheet" type="text/css" href="styles.css">
```

- **Estilo Embutido (Inline Style):** você também pode aplicar estilos diretamente a elementos HTML usando o atributo style. Isso é útil para estilos específicos que se aplicam apenas a um único elemento. Por exemplo:

```
<p style="color: blue; font-size: 16px;">Este é um parágrafo com estilo embutido.</p>
```

- **Estilo Interno (Embedded Style):** você pode incluir estilos CSS diretamente no cabeçalho do documento HTML, entre as tags <style>. Isso é útil quando os estilos se aplicam apenas àquela página específica. Por exemplo:

```
<style>

  body {

    font-family: Arial, sans-serif;

  }

</style>
```

Existem alguns benefícios e vantagens do processo de vinculação de folhas de estilo a documentos HTML. Ao compreender os benefícios desse processo, os desenvolvedores poderão otimizar a estrutura de seus projetos web, promover a reutilização de estilos, facilitar a manutenção do código e garantir uma experiência consistente e atraente para os usuários finais. Ao explorar cada vantagem em detalhes, será possível compreender como a vinculação eficiente de

folhas de estilo contribui para a eficácia e a qualidade do desenvolvimento web. Entre seus benefícios estão:

- **Reutilização de estilos:** ao vincular folhas de estilo externas, você pode reutilizar estilos em várias páginas, promovendo consistência e facilitando a manutenção.
- **Separação de preocupações:** separar o conteúdo HTML da apresentação CSS torna o código mais organizado e legível, facilitando a colaboração entre equipes de desenvolvimento.
- **Facilidade de manutenção:** ao fazer alterações nos estilos, você só precisa atualizar o arquivo CSS externo uma vez, e as mudanças serão refletidas em todas as páginas vinculadas a esse arquivo.

Ao longo desta aula, exploramos conceitos fundamentais do CSS, compreendemos a evolução do Módulo CSS3 e seu status atual, e aprendemos sobre a importância e os métodos de vinculação de folhas de estilo a documentos HTML. Ao finalizar este conteúdo, é evidente a relevância do CSS como uma linguagem essencial para a estilização e o design de páginas web, proporcionando controle sobre a apresentação e o layout do conteúdo. Compreender os conceitos básicos do CSS, estar atualizado sobre os recursos disponíveis no CSS3 e dominar as técnicas de vinculação de estilos são passos para o desenvolvimento eficaz e responsivo de websites. Ao aplicar esse conhecimento na prática, os desenvolvedores serão capazes de criar interfaces web visualmente atraentes, consistentes e adaptáveis, proporcionando uma experiência de usuário excepcional.

Vamos Exercitar?

Desenvolvimento responsivo – Fundamentos CSS e aplicações para o ambiente Web

Descrição da situação-problema:

Você decidiu criar sua primeira página web usando HTML e CSS para apresentar suas habilidades e interesses. Sua página inicial deve conter um cabeçalho com seu nome, uma seção de apresentação pessoal, uma seção de habilidades, e uma seção de projetos. O objetivo é aplicar os conceitos básicos de CSS para estilizar sua página de acordo com suas preferências.

Resolução da situação-problema:

Criar a estrutura básica do HTML com as tags:

- <link>, <body>, <header>, <main>, <section>, <footer>, <h1>, <h2>, <p>, , .
- Definir o conteúdo do cabeçalho <header> com seu nome dentro de um <h1>.

Criar as seções <section>:

- Para apresentação, habilidades e projetos, cada uma com um <h2> e conteúdo descrito dentro da tag <p> e .

Adicionar estilos de CSS:

- Definir o layout básico da página, incluindo margens, espaçamentos, cores e fontes.

Testar a página:

- Testar em diferentes dispositivos para garantir que a responsividade básica esteja funcionando corretamente.

Verificação:

- Verificar se todos os elementos estão alinhados corretamente e se o conteúdo é legível e visualmente agradável.

Ajustes:

- Fazer ajustes conforme necessário para melhorar a aparência e usabilidade da página.

O objetivo é chegar a uma página web como a apresentada a seguir:

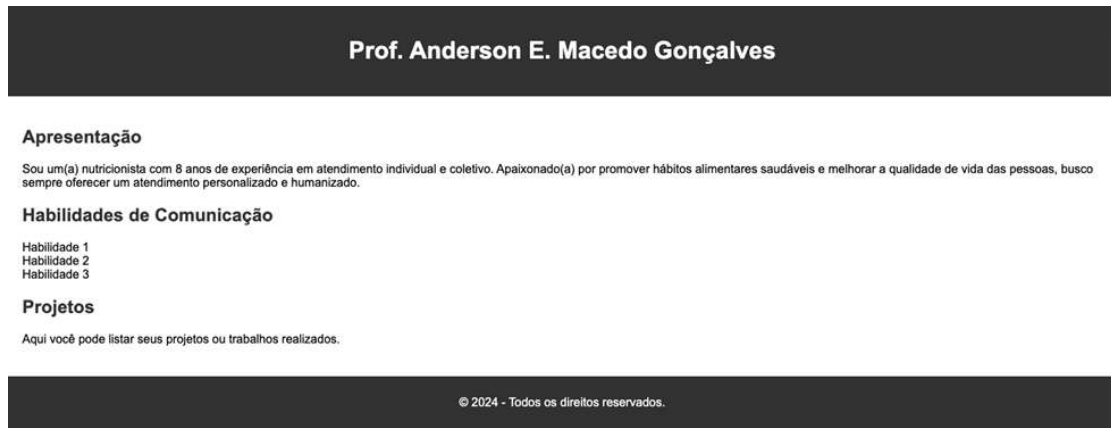


Figura 6 | Exemplo de página web.

Lembrando que a página é apenas uma sugestão de resolução da situação-problema. Para isso, lembre-se de que você terá que trabalhar com o arquivo HTML associando a ele um arquivo CSS, responsável pela estilização da página web.

A partir desse exemplo, você pode continuar explorando novos conceitos e técnicas, aprimorando suas habilidades e criando projetos cada vez mais complexos e sofisticados.

Saiba Mais

Para saber mais sobre os conceitos de folhas de estilos no que diz respeito às formas de apresentação desses conteúdos em páginas web, acesse o livro [Java para Web – Desenvolvimento de Aplicações](#), no repositório da Biblioteca Virtual. A indicação é para o estudo do Capítulo 5, páginas 50 a 57.

Bons estudos!

Referências Bibliográficas

ALVES, William P. Java para Web – **Desenvolvimento de Aplicações**. São Paulo: Saraiva, 2015. [Minha Biblioteca]

ALVES, William P. **Projetos de Sistemas Web Conceitos, Estruturas, Criação de Banco de dados e Ferramentas de Desenvolvimento**. São Paulo: Saraiva, 2015. [Minha Biblioteca]

FERREIRA, Arthur G. **Interface de programação de aplicações (API) e web services**. São Paulo: Saraiva, 2021. [Minha Biblioteca]

MARCOLINO, Anderson da S. **Frameworks Front End**. São Paulo: Saraiva, 2021. [Minha Biblioteca]

STRASSACAPA, Renata M. *et al.* **Web Analytics**. Porto Alegre: Sagah, 2022. [Minha Biblioteca]

ZABOT, Diego; MATOS, Ecivaldo de S. **Aplicativos com bootstrap e angular – como desenvolver apps responsivos**. São Paulo: Saraiva, 2020. [Minha Biblioteca]

Aula 2

PSEUDOCLASSE E SUAS FUNCIONALIDADES

Pseudoclasse e suas funcionalidades

Caro estudante,

Temos o prazer de convidar você para assistir à nossa videoaula, na qual exploraremos conceitos fundamentais de CSS que são essenciais para qualquer desenvolvedor web. Começaremos com as definições e a distinção entre pseudoclasses dinâmicas e estáticas, proporcionando a você uma compreensão sólida de como aplicar estilos que respondem de maneira inteligente às interações dos usuários.

Em seguida, abordaremos a seleção por elemento, classe e ID. Aprender a utilizar esses seletores permitirá que você estilize suas páginas com precisão, aplicando regras específicas a elementos individuais ou grupos de elementos, otimizando o design e a funcionalidade do seu site.

Por fim, exploraremos os agrupamentos, uma técnica importantíssima para aplicar estilos comuns a múltiplos elementos simultaneamente. Esta prática não só torna seu código mais eficiente, mas também melhora a manutenção e a consistência visual do seu projeto.

Esperamos ver você na videoaula!

Ponto de Partida

Olá, estudante. Seja bem-vindo a esta aula! Esperamos que esse conteúdo tenha valor agregado na construção do seu conhecimento.

Nesta aula, iniciaremos os estudos das folhas de estilo em cascata (CSS). Esta será uma oportunidade única para aprimorar suas habilidades e expandir seus conhecimentos em um dos pilares fundamentais do desenvolvimento web. Vamos explorar juntos conceitos essenciais, começando com as definições e a distinção entre pseudoclasses dinâmicas e estáticas. Compreender essas

bases é fundamental para criar estilos que respondem de maneira inteligente e eficiente às interações dos usuários.

Em seguida, nos aprofundaremos na seleção por elemento, classe e ID. Esses métodos de seleção são ferramentas poderosas que permitem estilizar suas páginas com precisão. A habilidade de aplicar estilos específicos a elementos determinados é uma competência indispensável para qualquer desenvolvedor web. Você aprenderá como utilizar seletores para maximizar o impacto visual e funcional de suas páginas, garantindo uma experiência de usuário rica e inovadora.

Por último, mas não menos importante, exploraremos os agrupamentos. Aprender a agrupar seletores permite aplicar estilos comuns a múltiplos elementos de forma simultânea, otimizando seu código e melhorando a manutenção e a legibilidade. Esta prática facilita o desenvolvimento e assegura a consistência visual em todo o seu projeto. Junte-se a nós nesta jornada de aprendizado, em que cada novo conceito contribuirá para transformar suas páginas web em interfaces profissionais e atraentes aos usuários.

Será apresentado o desenvolvimento de uma página básica utilizando os conceitos apresentados nesta aula. A solução terá o objetivo de vincular páginas HTML a uma única folha de estilo CSS.

A seguir vamos juntos aprender a dominar essas habilidades essenciais para o desenvolvimento web.

Vamos Começar!

O objetivo desta aula é apresentar a introdução aos conceitos fundamentais do *Cascading Style Sheets* (CSS). No desenvolvimento web, a função do CSS é separar a apresentação visual do conteúdo de um site, permitindo que os desenvolvedores definam e apliquem estilos consistentes e esteticamente agradáveis a elementos HTML. Com o CSS, é possível controlar a aparência de uma página, incluindo layout, cores, fontes, espaçamento e muito mais, de forma eficiente e organizada. Isso não só melhora a experiência do usuário ao tornar as

páginas mais atraentes e funcionais, como facilita a manutenção e atualização do código, promovendo uma estrutura limpa e modular.

O que são pseudoclasses?

Pseudoclasses são palavras-chave que podem ser adicionadas a seletores no CSS para especificar um estado especial de um elemento. Elas são precedidas por dois pontos (:) e são usadas para aplicar estilos com base em informações que não podem ser expressas usando seletores normais, como a posição de um elemento na árvore do documento, o estado de um elemento, ou a interação do usuário com a página. As pseudoclasses permitem aos desenvolvedores estilizar elementos de maneira que respondam a essas condições especiais sem precisar adicionar classes ou IDs extras ao HTML.

A seguir nos aprofundaremos em entender dois tipos de pseudoclasses: as dinâmicas e as estáticas. Você vai entender os seus objetivos e ver alguns exemplos práticos de como elas funcionam.

Pseudoclasses dinâmicas

As pseudoclasses dinâmicas são palavras-chave adicionadas aos seletores CSS para aplicar estilos baseados nas ações dos usuários ou nas mudanças de estado dos elementos durante a interação com a página. As mais comuns incluem **:hover**, **:focus**, e **:active**, cada uma desempenhando um papel específico na resposta às ações dos usuários.

A pseudoclasse **:hover** é utilizada para aplicar estilos a um elemento quando o usuário passa o cursor sobre ele. Este recurso é amplamente utilizado para fornecer feedback visual imediato, destacando botões, links, ou qualquer outro elemento interativo, o que melhora significativamente a navegabilidade. Por exemplo, ao passar

o cursor sobre um botão, ele pode mudar de cor, indicando ao usuário que o elemento é clicável. Observe:

```
button :hover {  
  
    background-color: blue;  
  
}
```

A pseudoclassee **:focus** é aplicada a um elemento quando ele está em foco, geralmente por meio de interações via teclado ou clique do mouse. Este é um aspecto essencial para a acessibilidade, pois permite que os usuários naveguem e interajam com a página utilizando o teclado. Um exemplo comum é a estilização de campos de formulário que mudam de cor ou borda quando recebem foco, ajudando os usuários a identificar em qual campo estão digitando, conforme demonstrado no exemplo a seguir:

```
input:focus {  
  
    border-color: green;  
  
}
```

Por último, a pseudoclassee **:active** é utilizada para aplicar estilos a um elemento quando ele está sendo clicado ou ativado. Esta pseudoclassee oferece feedback visual durante a interação, por

exemplo, escurecendo um botão enquanto ele está sendo pressionado:

```
a:active {  
  
    color: red;  
  
}
```

Com as explicações e exemplos demonstrados ficou claro que as pseudoclasses dinâmicas são elementos fundamentais no CSS, oferecendo uma maneira eficaz de estilizar elementos com base nas interações do usuário. Utilizando pseudoclasses como `:hover`, `:focus`, e `:active`, os desenvolvedores podem criar interfaces mais interativas e acessíveis, proporcionando feedback visual imediato e melhorando a experiência do usuário. Ao incorporá-las em seus projetos, os desenvolvedores podem transformar páginas estáticas em interfaces dinâmicas e responsivas, elevando a qualidade e a usabilidade dos sites.

O que são as seleções no CSS

Seleções no CSS são regras que determinam a quais elementos HTML um determinado conjunto de estilos deve ser aplicado. Elas são feitas utilizando seletores, que são padrões usados para identificar elementos específicos ou grupos de elementos na árvore do documento (DOM). Para melhor entender o DOM, recomendamos o seguinte link:

<https://tableless.github.io/iniciantes/manual/obasico/oquedom.html>.

Os seletores podem ser simples, como seletores de elementos, classes e IDs, ou mais complexos, como seletores combinados, pseudoclasses e pseudoelementos. Nesta aula vamos manter o nosso foco apenas nos seletores simples.

Seleção por elemento

O seletor elemento no CSS é um dos tipos de seletores mais simples e fundamentais. Ele seleciona todos os elementos HTML de um tipo específico e aplica um conjunto de estilos a eles. Os seletores de elementos são escritos usando o nome do elemento HTML diretamente, sem prefixos como pontos ou hashtags. Por exemplo, para estilizar todos os parágrafos (`<p>`), você usaria o seletor de elemento `p`.

O principal objetivo do seletor de elemento é aplicar estilos uniformes a todos os elementos de um determinado tipo em uma página web. Isso é extremamente útil para garantir consistência visual e estrutural. Ao usar seletores de elementos, os desenvolvedores podem rapidamente aplicar estilos a todos os cabeçalhos, parágrafos, listas, ou qualquer outro tipo de elemento HTML, sem a necessidade de adicionar classes ou IDs a cada um deles individualmente.

Vamos considerar um exemplo para ilustrar como o seletor de elemento funciona no desenvolvimento web. Suponha que você queira

estilizar todos os parágrafos (<p>) em uma página para que o texto seja azul e o tamanho da fonte seja 16 pixels:

HTML

```
<!DOCTYPE html>

<html lang=

<head>

  <meta charset=>

  <meta name= content=>

  <title>Exemplo de Seletor de Elemento</title>

  <link rel= href=>

</head>

<body>

<h1>Bem-vindo ao Meu Site</h1>

  <p>Este é o primeiro parágrafo.</p>

  <p>Este é o segundo parágrafo, com algum texto adicional.</p>

  <p>Este é o terceiro parágrafo.</p>

</body>

</html>
```

CSS (style.css)

```
p {
```

```
color: blue;  
  
font-size: 16px;  
  
}
```

Nesse exemplo, o seletor de elemento `p` é utilizado para selecionar todos os elementos `<p>` na página HTML. A seguir, são aplicados os estilos definidos no bloco CSS:

- **color: blue;** define a cor do texto como azul para todos os parágrafos.
- **font-size: 16px;** define o tamanho da fonte como 16 pixels para todos os parágrafos.

Ao aplicar esses estilos, todos os parágrafos na página HTML terão texto azul com tamanho de fonte de 16 pixels, proporcionando uma aparência uniforme e consistente sem a necessidade de classes ou IDs adicionais.

Dessa forma, o seletor de elemento é uma ferramenta essencial e básica no CSS, permitindo a aplicação de estilos uniformes a todos os elementos de um tipo específico em uma página web. Ele simplifica o desenvolvimento, assegurando consistência visual e estrutural de maneira eficiente. Utilizando seletores de elementos, os desenvolvedores podem rapidamente estilizar elementos HTML, melhorando a aparência e a usabilidade das páginas web.

Seleção por classe

O seletor de classe no CSS é uma ferramenta que permite aos desenvolvedores aplicar estilos a elementos específicos que têm uma determinada classe. As classes são atribuídas a elementos HTML usando o atributo `class`, e podem ser reutilizadas em múltiplos elementos, permitindo a aplicação de um conjunto de estilos a todos eles. No CSS, os seletores de classe são precedidos por um ponto (`.`),

seguido pelo nome da classe. Por exemplo, `.highlight` selecionaria todos os elementos com a classe "highlight".

O principal objetivo do seletor de classe é permitir a aplicação de estilos específicos a um grupo de elementos, independentemente de seu tipo. Isso oferece grande flexibilidade e reutilização de código, pois a mesma classe pode ser aplicada a diferentes tipos de elementos (como `<div>`, `<p>`, `<span>` etc.) para que todos recebam o mesmo estilo. Isso facilita a manutenção e a consistência visual do site, permitindo uma estilização granular e específica.

Observe um exemplo prático ilustrando como o seletor de classe funciona no desenvolvimento web. Suponha que você queira destacar

alguns elementos na página com um fundo amarelo e texto em negrito:

HTML

```
<!DOCTYPE html>
```

```
<html lang=>
```

```
<head>
```

```
  <meta charset=>
```

```
  <meta name= content=>
```

```
  <title>Exemplo de Seletor de Classe</title>
```

```
  <link rel=href=>
```

```
</head>
```

```
<body>
```

```
  <h1>Bem-vindo ao Meu Site</h1>
```

```
  <p class=>Este é o primeiro parágrafo, que está destacado.</p>
```

```
  <p>Este é o segundo parágrafo, que não está destacado.</p>
```

```
  <p class= >Este é o terceiro parágrafo, que também está destacado.  
</p>
```

```
  <div class=>Este é um div destacado.</div>
```

```
</body>
```

```
</html>
```

CSS (style.css)

```
.highlight {
```

```
background-color: yellow;  
  
font-weight: bold;  
  
}
```

Nesse exemplo, o seletor de classe `.highlight` é utilizado para selecionar todos os elementos que possuem a classe "highlight". Os estilos definidos no bloco CSS são:

- **background-color: yellow;** define o fundo dos elementos como amarelo.
- **font-weight: bold;** define o texto dos elementos como negrito.

Ao aplicar esses estilos, todos os elementos com a classe "highlight" na página HTML terão um fundo amarelo e texto em negrito. Isso inclui tanto os parágrafos quanto a `<div>`, demonstrando a flexibilidade do seletor de classe para estilizar diferentes tipos de elementos de maneira uniforme.

Assim, o seletor de classe no CSS é uma ferramenta que permite a aplicação de estilos específicos a grupos de elementos com uma determinada classe. Ele oferece grande flexibilidade e reutilização, facilitando a manutenção e garantindo a consistência visual do site. Utilizando seletores de classe, os desenvolvedores podem estilizar elementos por partes específicas, aprimorando a aparência e a usabilidade das páginas web de forma eficaz.

Siga em Frente...

Seleção por ID

O seletor por ID no CSS é uma ferramenta que permite aplicar estilos a um único elemento HTML que tem um identificador específico. IDs são únicos dentro de uma página, o que significa que cada ID deve ser atribuído a um único elemento. No CSS, os seletores de ID são

precedidos por uma hashtag (#), seguida pelo nome do ID. Por exemplo, #header selecionaria o elemento com o ID "header".

O principal objetivo do seletor por ID é aplicar estilos exclusivos e altamente específicos a um único elemento na página. Isso é útil quando é necessário garantir que certos estilos sejam aplicados somente a um elemento específico, sem afetar outros elementos. A utilização de IDs proporciona um nível mais alto de especificidade, garantindo que os estilos definidos tenham prioridade sobre aqueles aplicados por meio de seletores de elementos ou classes.

A seguir veremos um exemplo prático para ilustrar como o seletor por ID funciona no desenvolvimento web. Suponha que você queira

estilizar especificamente o cabeçalho da página com um fundo preto e um texto branco:

HTML

```
<!DOCTYPE html>

<html lang=>

<head>

  <meta charset=>

  <meta name= content=>

  <title>Exemplo de Seletor por ID</title>

  <link rel= href=>

</head>

<body>

  <header id=>

    <h1>Bem-vindo ao Meu Site</h1>

  </header>

  <p>Este é um parágrafo.</p>

  <p>Este é outro parágrafo.</p>

</body>

</html>
```

CSS (style.css)

```
#main-header {
```

```
background-color: black;  
  
color: white;  
  
padding: 20px;  
  
text-align: center;  
  
}
```

Nesse exemplo, o seletor por ID `#main-header` é utilizado para selecionar o elemento `<header>` com o ID "main-header". Os estilos definidos no bloco CSS são:

- `background-color: black;` define a cor de fundo do cabeçalho como preto.
- `color: white;` define a cor do texto do cabeçalho como branco.
- `padding: 20px;` adiciona um espaçamento interno de 20 pixels ao redor do conteúdo do cabeçalho.
- `text-align: center;` alinha o texto do cabeçalho ao centro.

Ao aplicar esses estilos, o cabeçalho específico da página terá um fundo preto, texto branco, espaçamento interno e texto centralizado, conforme definido. Esses estilos não afetarão outros elementos da página, garantindo que a estilização seja exclusiva para o cabeçalho.

Portanto, o seletor por ID no CSS é uma ferramenta bem específica, que permite aplicar estilos exclusivos a um único elemento em uma página web. Ele garante que certos estilos sejam aplicados de maneira precisa e prioritária, proporcionando um controle detalhado sobre a aparência de elementos únicos.

Agrupamentos

Agrupamento no CSS é uma técnica que permite aplicar o mesmo conjunto de estilos a vários seletores ao mesmo tempo. Em vez de escrever regras CSS separadas para cada seletor, os desenvolvedores podem agrupar seletores que compartilham os

mesmos estilos em uma única declaração. Isso torna o código CSS mais conciso, fácil de ler e manter, além de reduzir a repetição desnecessária de código.

O principal objetivo do agrupamento no CSS é simplificar a aplicação de estilos comuns a múltiplos elementos e melhorar a eficiência do código. Utilizando o agrupamento, os desenvolvedores podem:

1. **Reduzir redundância:** evitar a repetição de regras CSS para cada seletor individual, o que economiza tempo e esforço na escrita e manutenção do código.
2. **Melhorar a manutenção:** facilitar a atualização e manutenção do código CSS, já que mudanças em estilos comuns podem ser feitas em um único lugar.
3. **Aumentar a legibilidade:** tornar o código CSS mais limpo e organizado, o que ajuda outros desenvolvedores a entenderem rapidamente quais estilos são aplicados a quais elementos.

A seguinte imagem ilustra como o agrupamento funciona no CSS.
Suponha que você queira aplicar os mesmos estilos a todos os

cabeçalhos <h1>, <h2> e <h3> em uma página web.

SEM AGRUPAMENTO

```
h1 {  
  
  color: navy;  
  
  font-family: Arial, sans-serif;  
  
  margin-bottom: 10px;  
  
}
```

```
h2 {  
  
  color: navy;  
  
  font-family: Arial, sans-serif;  
  
  margin-bottom: 10px;  
  
}
```

```
h3 {  
  
  color: navy;  
  
  font-family: Arial, sans-serif;  
  
  margin-bottom: 10px;  
  
}
```

COM AGRUPAMENTO

```
h1, h2, h3 {  
  
    color: navy;  
  
    font-family: Arial, sans-serif;  
  
    margin-bottom: 10px;  
  
}
```

Nesse exemplo, o agrupamento permite combinar os seletores h1, h2 e h3 em uma única regra CSS. Os estilos definidos são:

- **color: navy;** define a cor do texto como azul marinho.
- **font-family:** Arial, sans-serif; define a família de fontes como Arial, com uma alternativa genérica sans-serif.
- **margin-bottom:** 10px; adiciona uma margem inferior de 10 pixels.

Ao usar o agrupamento, os mesmos estilos são aplicados a todos os cabeçalhos <h1>, <h2> e <h3>, tornando o código mais compacto e fácil de gerenciar.

Concluindo, o agrupamento no CSS é uma técnica eficaz para aplicar estilos comuns a múltiplos seletores, fazendo com que funcionem de maneira organizada. Além de ajudar a reduzir a redundância no código, ele melhora a manutenção e aumenta a legibilidade do CSS. Ao utilizar o agrupamento, os desenvolvedores podem escrever códigos que facilitarão tanto o desenvolvimento inicial quanto a manutenção contínua das páginas web.

Vamos Exercitar?

Desenvolvimento responsivo – Vinculação de folhas de estilos

Descrição da situação-problema:

Você está desenvolvendo um site para uma empresa de moda local e deseja aplicar estilos consistentes em todas as páginas. No entanto, você percebe que as regras de estilo estão se repetindo em vários arquivos HTML. Reflita: como você pode organizar e vincular de forma eficiente esses estilos para evitar repetições e facilitar a manutenção do código?

Considerando o cenário descrito, como você resolveria o problema de repetição de estilos em arquivos HTML ao vincular folhas de estilo a documentos de maneira eficiente?

Atividade: Desenvolva a vinculação de uma única folha de estilo a múltiplos documentos HTML no contexto da empresa de moda local. Descreva os passos necessários para organizar e vincular os estilos de forma a evitar repetições e garantir uma manutenção fácil do código.

Resolução da situação-problema (checklist):

1. **Organização dos arquivos:** comece organizando todos os estilos em um único arquivo CSS para garantir a centralização das regras de estilo.
2. **Identificação dos documentos HTML:** identifique todos os documentos HTML que fazem parte do site da empresa de moda.
3. **Vinculação da folha de estilo:** em cada documento HTML, adicione a tag <link> no cabeçalho (<head>) para vincular a folha de estilo única usando o atributo href.
4. **Estrutura da tag link:** certifique-se de que a estrutura da tag <link> esteja correta, apontando para o arquivo CSS correto usando o atributo href.
5. **Teste e validação:** realize testes para garantir que todas as páginas estejam exibindo corretamente e que não haja conflitos ou erros de formatação.
6. **Manutenção futura:** lembre-se de que, ao fazer alterações nos estilos, elas serão refletidas em todo o site devido à vinculação

única da folha de estilo. Certifique-se de documentar adequadamente as mudanças e revisar periodicamente o código para garantir sua integridade.

Para fazer o teste, basta criar dois arquivos HTML e vinculá-los ao mesmo CSS. Altere por exemplo a classe "h1{}" no CSS e veja se os dois arquivos HTML foram afetados. Se sim, você realizou o teste com sucesso.

Lembrando que essa maneira é apenas uma sugestão de resolução da situação-problema. Para isso, lembre-se de que você terá que trabalhar com o arquivo HTML associando a ele um arquivo CSS, que sempre será responsável pela estilização da página web.

A partir desse exemplo, você pode continuar explorando novos conceitos e técnicas, aprimorando suas habilidades e criando projetos cada vez mais complexos e sofisticados.

Saiba Mais

Para saber mais sobre pseudoclasses e seleção por elementos no que diz respeito à formas de apresentação desses conteúdos em páginas web, acesse o livro [*HTML & CSS: aprenda como construir páginas web*](#), no repositório da Biblioteca Virtual. A indicação é para o estudo do Capítulo 4, páginas 28 a 32.

Bons estudos!

Referências Bibliográficas

ALVES, William P. **HTML & CSS: aprenda como construir páginas web**. São Paulo: SRV LTDA, 2021. [Minha Biblioteca]

ALVES, William P. **Java para Web – Desenvolvimento de Aplicações**. São Paulo: Saraiva, 2015. [Minha Biblioteca]

ALVES, William P. **Projetos de Sistemas Web Conceitos, Estruturas, Criação de Banco de dados e Ferramentas de Desenvolvimento**. São Paulo: Saraiva, 2015. [Minha Biblioteca]

FERREIRA, Arthur G. **Interface de programação de aplicações (API) e web services**. São Paulo: Saraiva, 2021. [Minha Biblioteca]

MARCOLINO, Anderson da S. **Frameworks Front End**. São Paulo: Saraiva, 2021. [Minha Biblioteca]

STRASSACAPA, Renata M. *et al.* **Web Analytics**. Porto Alegre: Sagah, 2022. [Minha Biblioteca]

ZABOT, Diego; MATOS, Ecivaldo de S. **Aplicativos com bootstrap e angular – como desenvolver apps responsivos**. São Paulo: Saraiva, 2020. [Minha Biblioteca]

Aula 3

PRINCÍPIOS DO WEB DESIGN RESPONSIVO

Princípios do Web design responsivo

Caro estudante,

Que bom ter você por aqui. Desde já, convidamos você a assistir a essa videoaula, que visa proporcionar o contato com os princípios do web design responsivo entregando, de fato, um rico conhecimento nos conteúdos que serão abordados.

É necessário conhecer os fundamentos do web design responsivo no desenvolvimento web. Para tanto, esta aula foi preparada para guiar você por esses conceitos, apresentando técnicas e práticas que farão

you comprehend the creation of responsive layouts in web pages.

During the class, we will cover the basic concepts of responsive web design, but we will also explore principles for adaptation of layout and flexible design. With practical examples from the real world, you will have the opportunity to consolidate your understanding and apply these knowledge in your own projects.

Don't miss this chance to improve your skills and expand your knowledge in principles of responsive web design. Join us on this learning journey and strengthen your skills in the area. Your participation will be fundamental for the success of this educational experience!

Ponto de Partida

Hello, student. We welcome you to this class, and we hope that this content has value added in the construction of your knowledge.

In this starting point, our main objective is to provide you with a comprehension of the concept of responsive web design, or of responsiveness in the development of systems primarily with focus on the web. We will present the benefits and advantages of

adaptabilidade durante o processo para garantir uma experiência consistente ao usuário em diferentes dispositivos e tamanhos de tela.

Ao longo da aula, continuaremos explorando técnicas e estratégias para construir uma web verdadeiramente responsiva e adaptável. Você aprenderá a criar layouts flexíveis que se ajustam elegantemente a uma variedade de dispositivos, desde smartphones até desktops, garantindo uma experiência de usuário excepcional em qualquer contexto.

Então, prepare-se para embarcar conosco nesta jornada de exploração do web design responsivo. Estamos aqui para guiar e apoiar você em cada passo do caminho, partindo do princípio de que quanto mais você pratica uma atividade, melhor e mais rápido suas habilidades se desenvolvem.

E para tornar nosso aprendizado mais consistente, será apresentado um exemplo de uma página web utilizando HTML e CSS com alguns recursos responsivos, ou seja, a adequação da página ao dispositivo de acesso.

Assim, vamos iniciar a construção do seu aprendizado explorando o web design responsivo no cenário da programação web.

Bons estudos!

Vamos Começar!

O objetivo desta aula é apresentar a introdução à conceitos fundamentais do web design responsivo w suas principais características, além de iniciar você neste cenário tão importante do design de uma página web.

Compreendendo o conceito de web design responsivo

Web design responsivo é uma abordagem de design e desenvolvimento que garante que as páginas web se adaptem automaticamente a diferentes tamanhos de tela e dispositivos, proporcionando uma experiência de usuário otimizada em qualquer contexto. Isso é alcançado através do uso de técnicas como media queries, layouts fluidos e imagens flexíveis. A ideia central é criar um único site que funcione bem em desktops, tablets e smartphones, evitando a necessidade de desenvolver versões separadas para cada dispositivo.

A importância do web design responsivo tem crescido significativamente com o aumento do uso de dispositivos móveis. Atualmente, a maioria dos usuários acessa a internet por smartphones e tablets, e um site que não seja otimizado para esses dispositivos pode levar a uma experiência de usuário ruim, afastando potenciais visitantes e clientes. Além disso, motores de busca como o Google favorecem sites responsivos em seus algoritmos de ranking, o que significa que um design responsivo também pode melhorar a visibilidade e o tráfego orgânico de um site.

O papel do programador é crucial no desenvolvimento de um site responsivo. Além de implementar o design fornecido pelos designers, os programadores devem garantir que o site funcione corretamente em diferentes dispositivos e navegadores. Isso envolve a escrita de código CSS flexível e a utilização de media queries para ajustar o layout e os estilos com base no tamanho da tela. Os programadores também precisam otimizar o desempenho do site, garantindo que o tempo de carregamento seja rápido em todos os dispositivos, o que é essencial para a retenção de usuários.

A imagem a seguir ilustra simplificadaamente um site com web design responsivo:

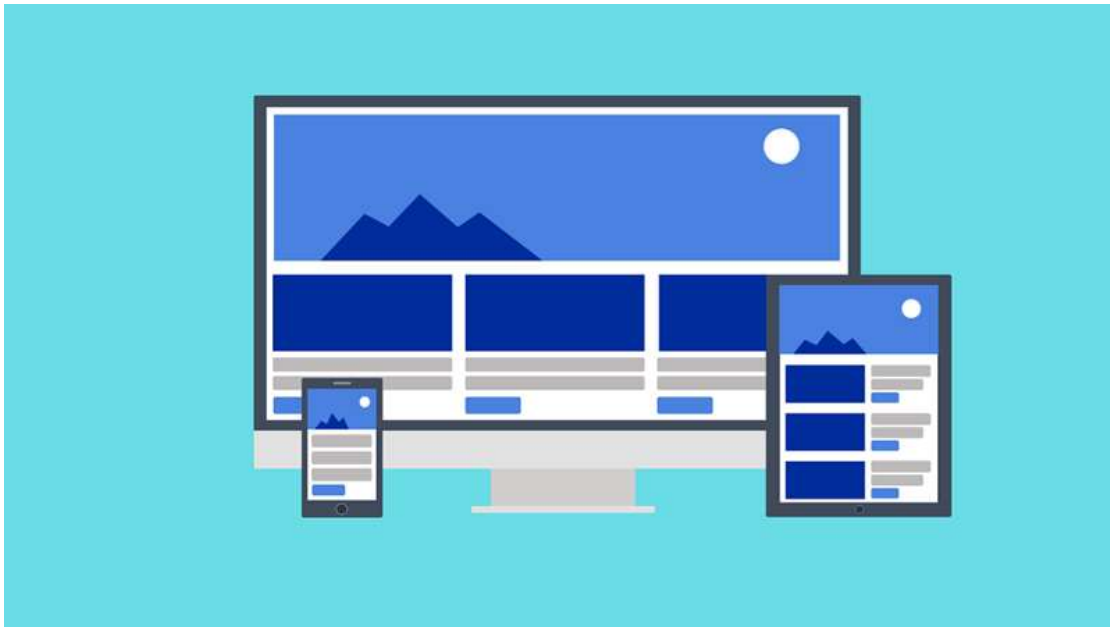


Figura 1 | Exemplo de um site com web design responsivo. Fonte: Pixabay.

Para criar um site verdadeiramente responsivo, a colaboração entre designers e programadores é essencial. Os programadores precisam entender as práticas recomendadas de design responsivo e estar familiarizados com as ferramentas e técnicas mais recentes, como frameworks CSS (por exemplo, Bootstrap) e metodologias de desenvolvimento mobile-first. Além disso, os programadores devem testar continuamente o site em vários dispositivos e resoluções de tela para identificar e corrigir problemas de usabilidade. Esse trabalho conjunto garante que o produto final seja acessível, eficiente e ofereça uma experiência de usuário consistente e agradável, independentemente do dispositivo utilizado.

Construindo uma web responsiva e adaptável

Construir um site com web design responsivo é um processo detalhado que envolve planejamento, design, desenvolvimento e testes contínuos. Neste tópico, vamos seguir um passo a passo que funcionará como um guia para auxiliar você, futuro programador, a criar um site que funcione bem em qualquer dispositivo.

1. Planejamento e design

Análise do público-alvo e dos dispositivos

O primeiro passo é entender quem são os usuários do site e quais dispositivos eles usam com mais frequência. Isso ajudará a definir prioridades de design e funcionalidades. Por exemplo, se a maioria dos usuários acessa o site via smartphone, é crucial otimizar a experiência móvel.

Wireframes e prototipagem

Crie wireframes e protótipos de alta fidelidade para diferentes tamanhos de tela. Ferramentas como Figma ou Adobe XD são úteis para visualizar como o layout se ajusta em telas pequenas, médias e grandes. Isso ajuda a identificar possíveis problemas de design antes de iniciar a codificação.

2. Criação de HTML limpo e semântica

Escreva HTML semântico, que melhora a acessibilidade e SEO (*Search Engine Optimization*). Use tags apropriadas como `<header>`, `<nav>`, `<main>`, `<section>`, e `<footer>` para definir a estrutura do documento.

Exemplo:

```
1  <!DOCTYPE html>
2  <html lang="pt-br">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-
      scale=1.0">
6      <title>Site Responsivo</title>
7      <link rel="stylesheet" href="styles.css">
8  </head>
9  <body>
10 <header>
11     <nav>
12         <ul>
13             <li><a href="#home">Home</a></li>
14             <li><a href="#about">Sobre</a></li>
15             <li><a href="#services">Serviços</a></li>
16             <li><a href="#contact">Contato</a></li>
17         </ul>
18     </nav>
19 </header>
20 <main>
21     <section id="home">
22         <h1>Bem-vindo ao Nosso Site</h1>
23     </section>
24     <!-- Outras seções -->
25 </main>
26 <footer>
27     <p>&copy; 2024 Empresa</p>
28 </footer>
29 </body>
30 </html>
```

Figura 2 | Exemplo de criação HTML limpo e semântica.

Este código HTML cria uma página web básica e responsiva com um cabeçalho contendo um menu de navegação, uma seção principal com um título de boas-vindas, e um rodapé com informações de copyright. Ele inclui um arquivo CSS externo para estilizar a página e ajusta o layout para diferentes tamanhos de tela através da meta tag viewport.

3. CSS Básico e definição de um layout flexível

Utilize unidades de medida flexíveis, como porcentagens, em vez de pixels fixos, para definir larguras e margens. O uso de max-width em vez de width também ajuda a tornar o layout mais adaptável.

Exemplo:

```
1  body {
2    font-family: Arial, sans-serif;
3    line-height: 1.6;
4  }
5
6  header, main, footer {
7    max-width: 1200px;
8    margin: 0 auto;
9    padding: 20px;
10 }
11
12 nav ul {
13   display: flex;
14   flex-wrap: wrap;
15   justify-content: space-around;
16   list-style: none;
17   padding: 0;
18 }
19
20 nav ul li a {
21   text-decoration: none;
22   color: #333;
23   padding: 10px;
24 }
25
26 section {
27   padding: 20px;
28 }
```

Figura 3 | Exemplo de layout flexível.

Este código estiliza elementos HTML para criar uma aparência limpa e organizada, com uma fonte consistente, espaçamento apropriado, e uma barra de navegação flexível e centralizada.

4. Implementar media queries para adaptar o layout

Use media queries para aplicar diferentes estilos baseados no tamanho da tela. Comece com um design mobile-first, aplicando estilos básicos para dispositivos móveis e utilizando media queries para adicionar melhorias para telas maiores.

Exemplo:


```
1  /* Estilos básicos para mobile-first */
2  body {
3      font-size: 16px;
4  }
5
6  nav ul {
7      flex-direction: column;
8      align-items: center;
9  }
10
11  /* Estilos para telas maiores que 600px */
12  @media (min-width: 600px) {
13      nav ul {
14          flex-direction: row;
15      }
16
17      section {
18          padding: 40px;
19      }
20  }
21
22  /* Estilos para telas maiores que 900px */
23  @media (min-width: 900px) {
24      body {
25          font-size: 18px;
26      }
27
28      nav ul li a {
29          padding: 15px;
30      }
31  }
```

Figura 4 | Exemplo de implementação de media queries.

Este código aplica estilos CSS adaptativos que se ajustam ao tamanho da tela: estilos básicos para dispositivos móveis, e ajustes para telas maiores que 600px e 900px, alterando a direção do menu, o tamanho da fonte e o espaçamento.

5. Utilizar imagens flexíveis e responsivas

Use atributos como `max-width: 100%` para garantir que as imagens se ajustem ao tamanho do contêiner. Utilize o elemento `<picture>` e o

atributo `srcset` para fornecer diferentes resoluções de imagem dependendo do tamanho da tela.

Exemplo:

```
<picture>
  <source media="(min-width: 800px)" srcset="imagem-grande.jpg">
  <source media="(min-width: 400px)" srcset="imagem-media.jpg">
  
</picture>
```

Figura 5 | Exemplo de utilização de imagens flexíveis e responsivas.

Este código exibe uma imagem que muda de acordo com o tamanho da tela: uma imagem grande para telas maiores que 800px, uma imagem média para telas entre 400px e 800px, e uma imagem pequena para telas menores que 400px.

6. Testar em diferentes dispositivos e navegadores

Teste continuamente o site em vários dispositivos e navegadores para garantir que o design seja responsivo. Ferramentas como o modo de visualização responsiva do Chrome DevTools podem simular diferentes tamanhos de tela, mas testes reais em dispositivos físicos são indispensáveis.

7. Otimizar para performance e acessibilidade

Garanta que o site carregue rapidamente em todas as condições, otimizando imagens e minimizando arquivos CSS e JavaScript. Utilize práticas de acessibilidade, como contrastes de cor adequados e suporte à navegação por teclado.

Diante de tudo o que vimos anteriormente, chegamos à conclusão de que construir um site com web design responsivo envolve uma combinação de planejamento cuidadoso, boas práticas de HTML e CSS, e testes rigorosos. Seguindo esses passos, os programadores podem criar sites que oferecem uma experiência de usuário excelente em qualquer dispositivo, atendendo às necessidades de um público

diversificado e maximizando o alcance e a eficácia do site na web moderna.

Explorando web design responsivo

O web design responsivo é fundamental na era digital atual, em que os usuários acessam websites por uma variedade de dispositivos com diferentes tamanhos de tela. A importância desse tipo de design reside na sua capacidade de proporcionar uma experiência de usuário otimizada, independentemente do dispositivo utilizado, o que pode aumentar o engajamento e a satisfação dos visitantes.

A seguir, abordaremos as principais vantagens na utilização de um sistema responsivo bem como as técnicas e ferramentas essenciais que podem ser utilizadas desse vasto campo da programação.

Siga em Frente...

Principais vantagens de ter um sistema responsivo

Para a equipe de desenvolvimento:

- **Manutenção simplificada:** com um único código base para todos os dispositivos, a manutenção e atualização do site são mais fáceis e eficientes.
- **Consistência de design:** um design responsivo garante que a aparência e a funcionalidade do site sejam consistentes em todas as plataformas, facilitando o desenvolvimento de novas funcionalidades e correções.
- **SEO melhorado:** motores de busca, como o Google, favorecem sites responsivos, melhorando a visibilidade e o ranking nos resultados de pesquisa.
- **Custo-efetividade:** desenvolver e manter um site responsivo é mais econômico do que criar versões separadas para desktop e dispositivos móveis.

Para o usuário e cliente:

- **Experiência de usuário melhorada:** um site responsivo adapta-se automaticamente ao dispositivo do usuário, proporcionando uma navegação mais agradável e intuitiva.
- **Acessibilidade aumentada:** usuários podem acessar o site de qualquer dispositivo, a qualquer momento, sem comprometer a funcionalidade ou a estética.
- **Redução na taxa de rejeição:** com uma melhor experiência de usuário, é menos provável que os visitantes abandonem o site rapidamente.
- **Maior alcance e engajamento:** a capacidade de alcançar usuários em diversos dispositivos aumenta o engajamento e potencialmente a conversão de visitantes em clientes.

Técnicas avançadas para aplicar a responsividade em páginas web

- **Media queries:** utilizar media queries no CSS para aplicar diferentes estilos baseados nas características do dispositivo conforme demonstrado a seguir na Figura 6, como largura da tela, altura, resolução e orientação.

```
1
2  @media (min-width: 768px) {
3      body {
4          background-color: lightblue;
5      }
6  }
7
```

Figura 6 | Exemplo de aplicação de Media Queries.

- **Layout fluido:** utilizar unidades relativas como porcentagens, em, rem e vw/vh para definir tamanhos de elementos, permitindo que eles se ajustem dinamicamente ao tamanho da tela. Observe na Figura 7:

```
1  ∨ .container {  
2    width: 80%;  
3    padding: 2rem;  
4  }
```

Figura 7 | Exemplo de aplicação de layout fluido.

- **Grid e Flexbox:** usar CSS Grid e Flexbox para criar layouts flexíveis e ajustáveis. Estes métodos permitem a fácil reorganização de elementos para diferentes tamanhos de tela.

```
1  ∨ .grid-container {  
2    display: grid;  
3    grid-template-columns: 1fr 1fr;  
4    gap: 10px;  
5  }  
6  
7  ∨ @media (max-width: 600px) {  
8    ∨ .grid-container {  
9      grid-template-columns: 1fr;  
10     }  
11  }
```

Figura 8 | Exemplo de aplicação de Grid.

- **Imagens responsivas:** utilizar o elemento `<picture>` e atributos `srcset` para servir imagens de diferentes resoluções conforme o tamanho da tela.

```
<picture>  
  <source media="(min-width: 800px)" srcset="large.jpg">  
  <source media="(min-width: 400px)" srcset="medium.jpg">  
    
</picture>
```

Figura 9 | Exemplo de aplicação do elemento `<picture>` e atributos `srcset`.

Ferramentas indicadas para desenvolvimento responsivo

1. **Bootstrap:** um dos frameworks CSS mais populares para o desenvolvimento de sites responsivos.

- Principais características: oferece um sistema de grid flexível, componentes predefinidos e utilitários CSS para criar rapidamente layouts responsivos. Suporta customização extensiva através de variáveis SASS.

2. **Foundation:** um framework front-end avançado, conhecido por sua flexibilidade e poder.

- Principais características: inclui um sistema de grid altamente personalizável, componentes modulares e utilitários CSS, além de suportar media queries semânticas.

3. **Visual Studio Code:** um editor de código-fonte poderoso e extensível, amplamente utilizado para desenvolvimento web.

- Principais características: suporte a uma vasta gama de extensões que melhoram o desenvolvimento responsivo, como emuladores de dispositivo, pré-visualização em tempo real, e integração com ferramentas de depuração e controle de versão.

Portanto, de acordo com o conteúdo abordado nesta aula, podemos concluir que o design responsivo é essencial para garantir que os sites ofereçam uma experiência de usuário otimizada em qualquer dispositivo. As técnicas avançadas de CSS, como media queries, layouts fluidos e imagens responsivas, combinadas com ferramentas assertivas como Bootstrap, Foundation, Figma e Visual Studio Code, facilitam a criação e manutenção de sites responsivos. Para a equipe de desenvolvimento, isso se traduz em eficiência e consistência, enquanto os usuários e clientes desfrutam de acessibilidade e uma experiência de navegação superior.

Vamos Exercitar?

Desenvolvimento responsivo – Princípios do web design responsivo

Descrição da situação-problema:

Você foi contratado por uma pequena empresa local para desenvolver o site institucional dela. Os responsáveis desejam que o site seja acessível e fácil de navegar tanto em computadores desktop quanto em dispositivos móveis, como smartphones e tablets. Sua tarefa é criar um layout simples e responsivo para o site, garantindo uma experiência de usuário consistente em diferentes telas.

Atividade:

Desenvolva um layout básico para o site da empresa, levando em consideração os princípios de construção de uma web responsiva e adaptável. Seu objetivo é criar um design que se ajuste automaticamente ao tamanho da tela do dispositivo em que está sendo visualizado, mantendo a usabilidade e a estética.

Resolução da situação-problema:

1. Comece definindo a estrutura básica do seu documento HTML, incluindo os elementos principais, como cabeçalho, navegação, conteúdo e rodapé.
2. Utilize as técnicas do CSS para criar estilos que se adaptem dinamicamente ao tamanho da tela. Experimente usar unidades relativas, como porcentagens e unidades flexíveis, para definir larguras, alturas e margens.
3. Teste seu layout em diferentes dispositivos e tamanhos de tela para garantir que ele se ajuste corretamente e mantenha a usabilidade em todas as condições.
4. Finalize seu projeto revisando e refinando os estilos conforme necessário para garantir uma aparência consistente e uma experiência de usuário agradável em todos os dispositivos.

O objetivo é chegar a uma página web como a apresentada a seguir:



Figura 10 | Exemplo de página web.

Lembrando que a página é apenas uma sugestão de resolução da situação-problema. Para isso, lembre-se de que você terá que trabalhar com o arquivo HTML associando a ele um arquivo CSS, responsável pela estilização da página web.

Ao final desta atividade, você estará mais preparado para enfrentar os desafios do desenvolvimento de sistemas, com uma compreensão mais sólida das técnicas necessárias para criar soluções digitais eficazes e adaptáveis às necessidades dos usuários modernos.

Saiba Mais

Para saber mais sobre os princípios do web design responsivo, suas características e particularidades, e como trabalhar com eles, acesse o livro [HTML & CSS: aprenda como construir páginas web](#), no repositório da Biblioteca Virtual. A indicação é para o estudo do Capítulo 5, páginas 33 a 36.

Bons estudos!

Referências Bibliográficas

ALVES, William P. **HTML & CSS: aprenda como construir páginas web**. São Paulo: SRV LTDA, 2021. [Minha Biblioteca]

ALVES, William P. **Java para Web – Desenvolvimento de Aplicações**. São Paulo: Saraiva, 2015. [Minha Biblioteca]

ALVES, William P. **Projetos de Sistemas Web Conceitos, Estruturas, Criação de Banco de dados e Ferramentas de Desenvolvimento**. São Paulo: Saraiva, 2015. [Minha Biblioteca]

FERREIRA, Arthur G. **Interface de programação de aplicações (API) e web services**. São Paulo: Saraiva, 2021. [Minha Biblioteca]

MARCOLINO, Anderson da S. **Frameworks Front End**. São Paulo: Saraiva, 2021. [Minha Biblioteca]

STRASSACAPA, Renata M. *et al.* **Web Analytics**. Porto Alegre: Sagah, 2022. [Minha Biblioteca]

ZABOT, Diego; MATOS, Ecivaldo de S. **Aplicativos com bootstrap e angular – como desenvolver apps responsivos**. São Paulo: Saraiva, 2020. [Minha Biblioteca]

Aula 4

MOBILE-FIRST E FERRAMENTAS DE WIREFRAME

Mobile-first e ferramentas de wireframe

Caro estudante,

Bem-vindo à mais uma emocionante jornada de aprendizado! Nesta videoaula, exploraremos temas essenciais para o desenvolvimento de interfaces digitais responsivas e adaptáveis. Prepare-se para conhecer o cenário do mobile-first e ferramentas de wireframe, conceitos fundamentais para quem busca criar experiências digitais incríveis.

Vamos trabalhar com o conceito do mobile-first, uma abordagem revolucionária no design de interfaces que prioriza a experiência do usuário em dispositivos móveis. Exploraremos os fundamentos dessa metodologia e como aplicá-la de forma eficaz em nossos projetos. Além disso, vamos nos aventurar pela prática, explorando diferentes técnicas e estratégias para otimizar o design para dispositivos móveis, garantindo uma experiência fluida e intuitiva para os usuários.

Não perca a oportunidade de aprimorar suas habilidades no design de interfaces digitais! Junte-se a nós nesta videoaula e embarque nessa jornada para contribuir com seu aprendizado. Independentemente do seu nível de experiência, este conteúdo será uma valiosa adição ao seu repertório profissional. Prepare-se para expandir seus horizontes e elevar suas habilidades no fascinante mundo do mobile-first e das ferramentas de wireframe.

Estamos ansiosos para compartilhar conhecimento e inspiração com você.

Ponto de Partida

Seja muito bem-vindo à próxima etapa da sua jornada de aprendizado! Nesta aula, mergulharemos em conceitos fundamentais do design responsivo, explorando o modelo mobile-first e as ferramentas de wireframe. Prepare-se para adquirir novos conhecimentos e habilidades que serão essenciais para a criação de experiências digitais incríveis e adaptáveis a diferentes dispositivos.

Para começar, vamos explorar o conceito do mobile-first, uma abordagem revolucionária no design de interfaces que prioriza a experiência do usuário em dispositivos móveis. Com o crescente uso de smartphones e tablets para acessar a internet, tornou-se fundamental projetar interfaces que sejam otimizadas para telas menores. Nesta parte da aula, entenderemos os fundamentos dessa metodologia e como ela influencia o processo de design, desde a estruturação do layout até a escolha de elementos que participarão da criação da interface web.

Também será dedicado um tempo para conhecer as ferramentas de wireframe, recursos essenciais no processo de design e exemplos de criação com a ferramenta Figma. Essa ferramenta permite criar protótipos visuais de forma rápida e eficiente, ajudando a visualizar a estrutura e o fluxo de uma página ou aplicativo.

Diante disso, será apresentado o layout inicial de uma página da web para dispositivos móveis com a técnica mobile-first. A página criada terá um contexto simples, porém cumprirá o objetivo de aplicação do conteúdo considerando o cenário.

Esteja pronto para uma jornada de aprendizado cheia de conteúdos que vão deixar sua aula e seu aprendizado mais interessantes! Estamos empolgados para compartilhar conhecimento e insights com você e ajudá-lo a se tornar um designer de interfaces ainda mais habilidoso.

Vamos começar?

Bons estudos!

Vamos Começar!

O objetivo desta aula é apresentar alguns conteúdos que estão na disciplina de desenvolvimento responsivo, no que diz respeito especificamente aos conteúdos granulares da aula, que são conceito de mobile-first, explorando o mobile-first e ferramentas de wireframe. A partir dos conteúdos abordados, serão apresentados exemplos de aplicação prática com o objetivo de fixação do conteúdo ao aprendizado do estudante.

Introdução

O mobile-first é uma abordagem de desenvolvimento web que ganhou destaque com o crescente uso de dispositivos móveis para acessar a internet. Nesta aula, exploraremos o conceito do mobile-first, seu contexto histórico, principais características e aplicações.

Com o avanço da tecnologia móvel e o aumento do número de usuários de smartphones e tablets, tornou-se essencial adaptar os sites para proporcionar uma experiência de usuário otimizada em dispositivos móveis. Anteriormente, muitos sites eram desenvolvidos primeiro para desktops e depois adaptados para dispositivos móveis,

o que muitas vezes resultava em páginas lentas e difíceis de usar em telas menores.

O que é mobile-first?

O mobile-first é uma abordagem de design e desenvolvimento web que prioriza a criação de layouts e funcionalidades para dispositivos móveis antes de adaptá-los para desktops. Isso significa começar o processo de design e desenvolvimento com o foco na experiência do usuário em dispositivos móveis e, em seguida, expandir para dispositivos maiores, como desktops e laptops.

Principais características e utilizações

Dentre as características da abordagem de desenvolvimento mobile-first, se destacam algumas delas, além de suas utilizações. A seguir elas são citadas e explicadas:

- **Layout responsivo:** os layouts desenvolvidos com a abordagem mobile-first são responsivos e se ajustam automaticamente a diferentes tamanhos de tela, proporcionando uma experiência consistente em dispositivos móveis e desktops.
- **Performance aprimorada:** ao priorizar o desenvolvimento para dispositivos móveis, é possível reduzir o tamanho dos arquivos e otimizar o desempenho do site, resultando em tempos de carregamento mais rápidos e uma experiência de usuário mais fluida.
- **Foco na usabilidade:** o mobile-first incentiva os designers e desenvolvedores a simplificar os elementos da interface e priorizar o conteúdo mais importante para os usuários móveis, garantindo uma navegação fácil e intuitiva.

Aplicações do mobile-first

- **Websites corporativos:** empresas de todos os setores estão adotando o mobile-first para garantir que seus sites ofereçam

uma experiência de usuário consistente em todos os dispositivos.

- **E-commerce:** as lojas on-line estão priorizando o mobile-first para garantir que os clientes possam navegar, pesquisar e fazer compras facilmente em seus dispositivos móveis.
- **Aplicativos web progressivos (PWAs):** os PWAs são aplicativos web que oferecem funcionalidades semelhantes a aplicativos nativos e são desenvolvidos com a abordagem mobile-first para garantir compatibilidade e desempenho ideais em dispositivos móveis.

O mobile-first é mais do que uma simples tendência; é uma abordagem essencial para o design e desenvolvimento web na era dos dispositivos móveis. Ao priorizar a experiência do usuário em dispositivos móveis desde o início do processo de design, os desenvolvedores podem criar sites responsivos, de alto desempenho e com foco na usabilidade, proporcionando uma experiência de usuário excepcional em todos os dispositivos.

Explorando o mobile-first

Até o momento, discutimos o conceito do mobile-first e sua importância no design e desenvolvimento web. A partir de agora, aprofundaremos nosso conhecimento explorando as práticas do mobile-first e como implementá-lo em projetos reais. Para tanto, vamos apresentar algumas características e propriedades:

Layout responsivo:

- **Media queries:** utilize media queries para definir pontos de interrupção onde o layout do site será ajustado para diferentes tamanhos de tela.

```
1  /* Exemplo de media query para dispositivos móveis */
2  √ @media only screen and (max-width: 600px) {
3      /* Estilos para dispositivos móveis */
4      √ body {
5          font-size: 14px;
6      }
7  }
```

Figura 1 | Media Queries.

Imagens e mídia:

- **Imagens responsivas:** utilize a propriedade `max-width: 100%`; em imagens para garantir que elas se ajustem automaticamente ao tamanho da tela.

```
2  /* Estilo para tornar imagens responsivas */
3  √ img {
4      max-width: 100%;
5      height: auto;
6  }
```

Figura 2 | Max-width.

Priorização de conteúdo:

- **Ordem dos elementos:** considere a ordem em que os elementos são dispostos no HTML para garantir que o conteúdo mais importante seja apresentado primeiro em dispositivos móveis.

```
/* Exemplo de HTML com conteúdo mais importante primeiro */
√ <div class="header">
    <h1>Título Principal</h1>
    <p>Descrição do conteúdo principal.</p>
</div>
```

Figura 3 | Ordem dos elementos.

Navegação simplificada:

- **Menu hambúrguer:** utilize menus hambúrguer para economizar espaço e oferecer uma experiência de navegação mais intuitiva em dispositivos móveis.

```

1  <!-- Exemplo de menu hambúrguer -->
2  <div class="menu-toggle">
3      <input type="checkbox" />
4      <span></span>
5      <span></span>
6      <span></span>
7  <ul class="menu">
8      <li><a href="#">Link 1</a></li>
9      <li><a href="#">Link 2</a></li>
10     <li><a href="#">Link 3</a></li>
11 </ul>
12 </div>

```

Figura 4 | Menu hambúrguer.

Exemplo prático:

Vamos criar um layout simples usando a abordagem do mobile-first. A seguir está um exemplo básico de HTML e CSS para um site responsivo. A Figura 5 representa o HTML.

```

index.html
1  <!DOCTYPE html>
2  <html lang="en">
3  <head>
4      <meta charset="UTF-8">
5      <meta name="viewport" content="width=device-width, initial-scale=1.0">
6      <title>Exemplo Mobile-first</title>
7      <link rel="stylesheet" href="style.css">
8  </head>
9  <body>
10     <header>
11         <h1>Meu Site Responsivo</h1>
12         <nav>
13             <ul>
14                 <li><a href="#">Início</a></li>
15                 <li><a href="#">Sobre</a></li>
16                 <li><a href="#">Contato</a></li>
17             </ul>
18         </nav>
19     </header>
20     <main>
21         <section>
22             <h2>Sobre</h2>
23             <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit. Vivamus non libero sit amet odio condimentum finibus eu eu ex.</p>
24         </section>
25         <section>
26             <h2>Serviços</h2>
27             <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit. Vivamus non libero sit amet odio condimentum finibus eu eu ex.</p>
28         </section>
29         <section>
30             <h2>Contato</h2>
31             <p>Lorem ipsum dolor sit amet, consectetur adipiscing elit. Vivamus non libero sit amet odio condimentum finibus eu eu ex.</p>
32         </section>
33     </main>
34     <footer>
35         <p>&copy; 2024 Meu Site Responsivo</p>
36     </footer>
37 </body>
38 </html>

```

Figura 5 | HTML.

Já a Figura 6 representa o código CSS para o HTML da Figura 5.



```
1  /* Estilos gerais */
2  body {
3      font-family: Arial, sans-serif;
4      margin: 0;
5      padding: 0;
6  }
7
8  header {
9      background-color: #333;
10     color: #fff;
11     padding: 10px;
12 }
13
14 nav ul {
15     list-style-type: none;
16     padding: 0;
17 }
18
19 nav ul li {
20     display: inline;
21     margin-right: 20px;
22 }
23
24 nav ul li:last-child {
25     margin-right: 0;
26 }
27
28 main {
29     padding: 20px;
30 }
31
32 section {
33     margin-bottom: 20px;
34 }
35
36 footer {
37     background-color: #333;
38     color: #fff;
39     text-align: center;
40     padding: 10px;
41 }
```

Figura 6 | CSS.

Explorar o conceito do mobile-first é essencial para criar sites responsivos e adaptáveis aos dispositivos móveis, proporcionando uma experiência de usuário aprimorada. Ao seguir as práticas e exemplos apresentados nesta aula, você estará preparado para desenvolver projetos web com foco na mobilidade e usabilidade.

Siga em Frente...

Ferramentas de wireframe

A partir de agora vamos explorar as ferramentas de wireframe, começando compreendendo o contexto histórico e a importância do wireframe no processo de design de interface, além de discutir suas principais características e utilizações. Em seguida, examinaremos algumas das ferramentas mais populares disponíveis no mercado para criar wireframes eficientes e visualmente atraentes.

Mas o que é wireframe?

Um wireframe é uma representação visual básica de uma página da web ou de uma aplicação, mostrando a estrutura, o layout e a disposição dos elementos semelhante a um esboço. Ele serve como um esqueleto ou plano inicial do projeto, permitindo aos designers e desenvolvedores visualizar a organização geral e o fluxo de informações antes de iniciar o design detalhado.

Falando um pouco do histórico desse tipo de ferramenta, os wireframes têm suas raízes na arquitetura e no design gráfico, áreas em que esboços e maquetes são usados para visualizar e comunicar conceitos de design. No contexto digital, os wireframes tornaram-se uma ferramenta indispensável no desenvolvimento de interfaces de usuário, facilitando a colaboração entre equipes de design e desenvolvimento e garantindo a usabilidade e acessibilidade das aplicações.

Em alguns cursos superiores de desenvolvimento de software, os professores chegam a institucionalizar alguma ferramenta de

wireframe para que os discentes façam um protótipo do projeto.

Algumas de suas principais características e utilizações são:

- **Simplicidade:** wireframes são simples e esquemáticos, focando a estrutura e layout sem se preocupar com detalhes de design.
- **Comunicação visual:** eles servem como uma linguagem comum para comunicar ideias e conceitos entre designers, desenvolvedores e partes interessadas.
- **Validação de conceitos:** wireframes permitem testar e validar diferentes abordagens de design antes de investir tempo e recursos no desenvolvimento completo.
- **Iteração rápida:** como são rápidos e fáceis de criar, os wireframes facilitam a iteração e refinamento do design durante o processo de desenvolvimento.

Agora que já conhecemos um pouco das características dos wireframes, vamos conhecer algumas ferramentas do mercado que apoiam o processo da construção da interface com o usuário.

Balsamiq Wwreframes: uma ferramenta simples e intuitiva, ideal para esboçar rapidamente ideias de design. Com essa ferramenta é possível desenvolver uma interface simples e intuitiva, com elementos pré-definidos para agilizar o processo de criação e ainda contar com possibilidades de exportar para diferentes formatos de arquivo.

Para obter o Balsamiq e desfrutar dos seus benefícios é muito simples, basta acessar a [balsamiq](https://balsamiq.com), que contém o endereço oficial da ferramenta e fazer o download. Algumas das opções de licenças são:

- Balsamiq Cloud Trial.
- Desktop Trial.

Infelizmente a ferramenta é paga, entretanto, há outras ferramentas que são muito eficientes, como o Miro, e permitem a criação de contas gratuitas para que você trabalhe com uma quantidade limitada de recursos. É muito interessante para aprender a ferramentas e testar o seu potencial. A Figura 7 apresenta interfaces para criação de modelos no Balsamiq wireframes:



Figura 7 | Balsamiq wireframes. Fonte: Balsamiq Wireframes [s. d.].

Figma: uma ferramenta baseada em nuvem, que permite a colaboração em tempo real e é adequada para equipes distribuídas. O Figma foi lançado em 2016 como uma ferramenta de design baseada em nuvem, desenvolvida para facilitar a colaboração e o trabalho em equipe em projetos de design de interface. Suas principais características são:

- Baseado em nuvem.
- Colaboração em tempo real.
- Design responsivo.
- Componentes reutilizáveis.
- Prototipagem interativa.

Para começar a usar o Figma, basta criar uma conta gratuita no site oficial [figma](https://figma.com), ou logar com sua conta do Google e começar a explorar a interface e os recursos disponíveis. O Figma oferece uma ampla variedade de tutoriais e recursos de aprendizado para ajudar os usuários a se familiarizarem com a plataforma e a aproveitarem ao máximo suas capacidades. A Figura 8 apresenta interfaces para criação de modelos no Figma:

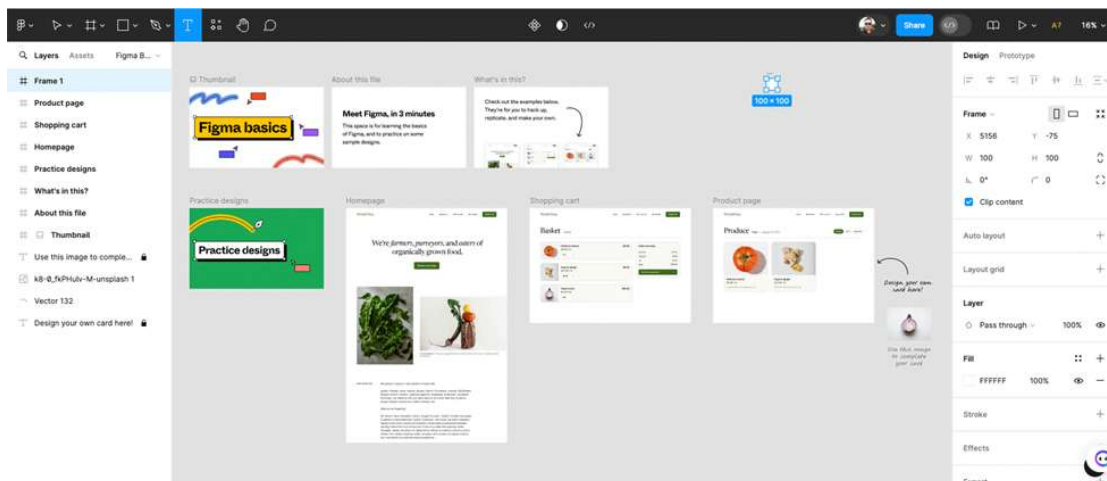


Figura 8 | Figma. Fonte: captura de tela do Figma.

O Figma é uma ferramenta poderosa e flexível para design de interfaces, incluindo wireframes e protótipos. Sua interface intuitiva, recursos avançados e capacidade de colaboração em tempo real o tornam uma escolha popular entre designers e equipes de design em todo o mundo. Ao dominar o Figma, você estará preparado para criar designs eficientes e colaborativos que atendam às necessidades de seus projetos de design.

Nesta aula, exploramos os conceitos fundamentais do design responsivo, abordando o mobile-first, as ferramentas de wireframe e o uso do Figma como uma poderosa plataforma de design colaborativo. Ao compreender o mobile-first, entendemos a importância de priorizar a experiência do usuário em dispositivos móveis e como isso influencia o processo de design. As ferramentas de wireframe, incluindo o Figma, nos fornecem recursos avançados para criar layouts adaptáveis e protótipos interativos, facilitando a validação de ideias e o desenvolvimento de designs eficientes.

Ao dominar esses conceitos e ferramentas, estamos preparados para enfrentar os desafios do design responsivo e criar experiências digitais impactantes e acessíveis em uma variedade de dispositivos e contextos. Portanto, com a prática e a experimentação contínua, podemos aprimorar nossas habilidades e elevar nossos projetos de design a novos patamares de excelência.

Vamos Exercitar?

Desenvolvimento responsivo – Mobile-first e ferramentas de wireframe

Descrição da situação-problema:

Você é um estudante de design de interfaces digitais e está iniciando seus estudos sobre o conceito de mobile-first. Seu professor solicitou que você aplique esse conceito em um projeto simples para entender melhor como funciona. Como parte desse projeto, você precisa desenvolver o layout inicial de uma página da web considerando a abordagem mobile-first.

Enunciado: Considerando que você está começando a explorar o conceito de mobile-first, sua tarefa é criar o layout inicial de uma página da web para dispositivos móveis. Você deve priorizar a experiência do usuário em telas menores, garantindo que o conteúdo seja acessível e fácil de navegar em smartphones e tablets.

Checklist:

- Definição do objetivo do projeto.
- Criação de um wireframe básico.
- Priorização do conteúdo essencial.
- Adaptação do layout para dispositivos móveis.
- Teste da experiência do usuário em dispositivos móveis.

Resolução da situação-problema:

1. **Defina o objetivo do projeto:** antes de começar a projetar, identifique qual será o propósito da página da web e quais informações ou funcionalidades serão priorizadas para os usuários de dispositivos móveis.
2. **Crie um wireframe:** utilize papel e lápis ou ferramentas de wireframe on-line para esboçar o layout inicial da página. Foque elementos essenciais, como o cabeçalho, menu de navegação, conteúdo principal e rodapé.

3. **Priorize o conteúdo:** lembre-se de que em um design mobile-first, é crucial priorizar o conteúdo mais importante e eliminar elementos desnecessários. Certifique-se de que o conteúdo principal seja facilmente acessível e compreensível em telas menores.
4. **Adapte o layout para dispositivos móveis:** ao desenvolver o design, leve em consideração as especificidades de telas menores, como a necessidade de menus de navegação simplificados, botões de fácil acesso e conteúdo otimizado para rolagem vertical.
5. **Teste o layout em dispositivos móveis:** após criar o layout inicial, teste-o em diferentes dispositivos móveis para garantir que a experiência do usuário seja consistente e intuitiva em telas de diferentes tamanhos e resoluções.

O objetivo é chegar a uma página web como a apresentada a seguir:



Lembrando que a página é apenas uma sugestão de resolução da situação-problema. Para isso, lembre-se de que você terá que trabalhar com o arquivo HTML associando a ele um arquivo CSS, responsável pela estilização da página web. Você pode alterar o template sem problema algum, utilizando sua criatividade e dando vida a seu projeto.

A partir desse exemplo, você pode continuar explorando novos conceitos e técnicas, aprimorando suas habilidades e criando projetos cada vez mais complexos e sofisticados.

Saiba Mais

Para saber mais sobre wireframes, suas características e ferramentas, acesse o livro [Design de Navegação Web](#), no repositório da Biblioteca Virtual. A indicação é para o estudo do Capítulo 9, páginas 260 a 287. Bons estudos!

Referências Bibliográficas

ALVES, William P. **Java para Web – Desenvolvimento de Aplicações**. São Paulo: Saraiva, 2015. [Minha Biblioteca]

ALVES, William P. **Projetos de Sistemas Web Conceitos, Estruturas, Criação de Banco de dados e Ferramentas de Desenvolvimento**. São Paulo: Saraiva, 2015. [Minha Biblioteca]

BALSAMIQ WIREFRAMES. Página inicial. Balsamiq Wireframes, [s. d.]. Disponível em: <https://balsamiq.com/wireframes/>. Acesso em: 14 dez. 2024.

FERREIRA, Arthur G. **Interface de programação de aplicações (API) e web services**. São Paulo: Saraiva, 2021. [Minha Biblioteca]

KALBACH, James. **Design de navegação web**. Porto Alegre: Grupo A, 2009. [Minha Biblioteca]

MARCOLINO, Anderson da S. **Frameworks Front End**. São Paulo: Saraiva, 2021. [Minha Biblioteca]

STRASSACAPA, Renata M. *et al.* **Web Analytics**. Porto Alegre: Sagah, 2022. [Minha Biblioteca]

ZABOT, Diego; MATOS, Ecivaldo de S. **Aplicativos com bootstrap e angular – como desenvolver apps responsivos**. São Paulo: Saraiva, 2020. [Minha Biblioteca]

Encerramento da Unidade

RESPONSIVIDADE E ADAPTABILIDADE NA WEB

Videoaula de Encerramento

Olá, estudante! Seja muito bem-vindo à nossa videoaula especial sobre responsividade e adaptabilidade na web. Exploraremos fundamentos essenciais que impactam diretamente a prática profissional de quem atua no desenvolvimento front-end com a programação web.

Diante de um cenário cada vez mais conectado e diversificado em termos de dispositivos, é necessário para qualquer profissional de desenvolvimento web compreender os fundamentos da responsividade e adaptabilidade de seus produtos, ou seja, de seus sistemas e aplicações desenvolvidas. Nesta videoaula, exploraremos como garantir que nossos sites e aplicativos sejam acessíveis e utilizáveis em uma variedade de dispositivos, desde smartphones até desktops.

Além disso, discutiremos estratégias e técnicas para criar layouts que se ajustem dinamicamente ao tamanho da tela, garantindo uma apresentação visual atraente e funcional.

Não importa se você está dando os primeiros passos na área ou buscando aprimorar suas habilidades, esta videoaula oferecerá insights valiosos de como tornar seus projetos web responsivos e adaptáveis.

Então, não perca tempo! Aperte o play, absorva o conhecimento e embarque nessa jornada emocionante pelo mundo da programação web. Assista agora e dê mais um passo em busca do conhecimento.

Ponto de Chegada

Olá, estudante! Para desenvolver a competência desta Unidade, que é conhecer os princípios do design responsivo, utilizando os fundamentos do CSS e pseudoclasses, para criar layouts adaptáveis em diferentes dispositivos, você deverá primeiramente conhecer os conceitos fundamentais abordados durante a unidade de ensino.

Uma das competências essenciais é compreender inicialmente do que se trata a responsividade, seu real significado e como ela pode auxiliar na criação de interfaces amigáveis para o usuário.

Ao compreender os princípios do design responsivo, utilizando os fundamentos do CSS e pseudoclasses, você adquire habilidades para criar layouts adaptáveis em diferentes dispositivos. Essa competência permite desenvolver uma abordagem analítica e estruturada na resolução de problemas de design, transformando desafios complexos em soluções eficientes.

Com o avanço do conteúdo, você aprende como avaliar e aplicar o conceito de responsividade em diversos contextos, partindo da criação, até a manipulação de elementos dentro de uma página web.

O entendimento dos fundamentos dessa teoria incentiva o desenvolvimento de um layout agradável e harmônico para execução

do sistema. Você aprende a analisar de maneira criteriosa, como dispor os diversos componentes utilizando recursos do CSS atingindo uma melhor experiência do usuário.

Essas habilidades são essenciais para a construção de websites adaptáveis, e para você se destacar profissionalmente em um mercado cada vez mais centrado na experiência do usuário.

É Hora de Praticar!

Convidamos você, estudante, a colocar em prática o conhecimento adquirido nesta unidade de ensino. Para isso, vamos abordar um estudo de caso com o cenário descrito a seguir:

Você deve criar uma página aplicando os princípios da responsividade da web. A página deve conter dois blocos retangulares, um na cor azul e outro na cor vermelha respectivamente, os dois, dispostos inicialmente em tela cheia. No centro dos dois blocos deve aparecer escrito na cor branca o nome da disciplina: **desenvolvimento responsivo**.

Conforme o tamanho da tela é reduzido, os blocos devem se ajustar responsivamente, diminuindo proporcionalmente em tamanho, garantindo uma experiência de usuário consistente em diferentes dispositivos e tamanhos de tela.

O desafio a ser superado é justamente a adequação dos blocos e textos ao tamanho da interface.

Desenvolva a solução utilizando apenas HTML e CSS, seguindo os princípios de design responsivo, garantindo:

- Experiência consistente.
- Legibilidade aprimorada.
- Navegação intuitiva.
- Acessibilidade melhorada.

Refleta

Fundamentos do CSS

Dominar os fundamentos do CSS possibilita a criação de designs esteticamente atraentes e fomenta uma compreensão mais profunda da estruturação e estilização de elementos na web. Reflita a respeito de como a compreensão desses conceitos básicos pode influenciar sua capacidade de criar interfaces intuitivas e funcionais: Como a aplicação dos princípios do CSS pode melhorar a usabilidade e acessibilidade de um site?

Layout acessível

Ao explorar os fundamentos do CSS, é essencial considerar não a estética visual, além da adaptabilidade e responsividade do design em diferentes dispositivos e tamanhos de tela. Pergunte a si mesmo: Como a compreensão do conceito de layout pode impactar a experiência do usuário em dispositivos móveis e desktops? Como posso garantir que meu design seja acessível e eficaz em qualquer contexto de navegação?

Resolução do estudo de caso

Sugestão de solução

1.Estruturação do HTML:

- Verifique se o HTML contém os elementos necessários para os blocos azul e vermelho, dentro de um contêiner.

2.Estilo base do CSS:

- Defina o estilo base do HTML e do corpo da página, removendo margens e preenchimentos padrão.

3.Estilo do contêiner:

- Use flexbox para criar um contêiner flexível que ocupará a altura total da viewport.

4. Estilo dos blocos azul e vermelho:

- Defina as propriedades CSS para os blocos azul e vermelho, tornando-os flexíveis e proporcionais ao tamanho da tela.

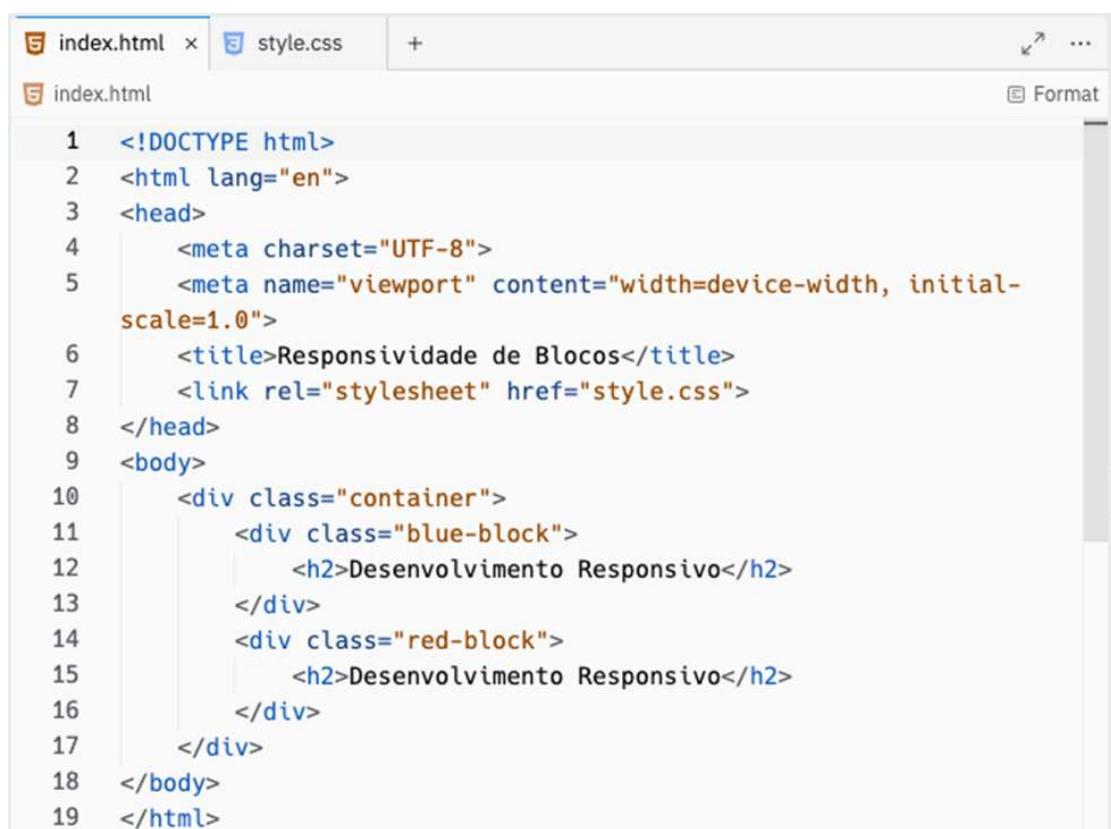
5. Responsividade:

- Utilize a media query @media para aplicar regras CSS específicas quando a largura da tela for menor que 768px.
- No media query, altere a direção do contêiner de linha para coluna, garantindo que os blocos se ajustem corretamente em dispositivos menores.

6. Teste e ajuste:

- Verifique se a página se comporta conforme o esperado em diferentes tamanhos de tela, testando a responsividade dos blocos azul e vermelho.
- Faça ajustes conforme necessário para garantir uma experiência de usuário consistente e agradável.

Sugestão para codificação dos arquivos HTML



```
1 <!DOCTYPE html>
2 <html lang="en">
3 <head>
4   <meta charset="UTF-8">
5   <meta name="viewport" content="width=device-width, initial-
  scale=1.0">
6   <title>Responsividade de Blocos</title>
7   <link rel="stylesheet" href="style.css">
8 </head>
9 <body>
10   <div class="container">
11     <div class="blue-block">
12       <h2>Desenvolvimento Responsivo</h2>
13     </div>
14     <div class="red-block">
15       <h2>Desenvolvimento Responsivo</h2>
16     </div>
17   </div>
18 </body>
19 </html>
```

Sugestão para codificação dos arquivos CSS

```
index.html style.css x +
style.css > .red-block Format
1  body {
2      margin: 0;
3      padding: 0;
4      box-sizing: border-box;
5  }
6
7  .container {
8      display: flex;
9      height: 100vh;
10 }
11
12 .blue-block, .red-block {
13     flex: 1;
14     display: flex;
15     justify-content: center;
16     align-items: center;
17     color: white;
18     font-size: 4em;
19     font-family: Arial, sans-serif;
20     transition: flex 0.5s ease-in-out, font-size 0.5s ease-in-out;
21     overflow: hidden; /* Evita que o texto ultrapasse o contêiner */
22     text-align: center; /* Centraliza o texto horizontalmente */
23 }
24
25 .blue-block {
26     background-color: blue;
27 }
28
29 .red-block {
30     background-color: red;
31 }
32 @media screen and (max-width: 1100px) {
33     .container {
34         flex-direction: column;
35     }
36     .blue-block, .red-block {
37         font-size: 2em;
38     }
39 }
```

Exemplo de visualização



Conclusão

A implementação bem-sucedida da responsividade dos blocos azul e vermelho em uma página web de exemplo demonstra a importância do design responsivo no desenvolvimento de interfaces web modernas. Ao utilizar técnicas como flexbox e media queries em CSS, conseguimos criar uma experiência de usuário fluida e consistente em uma variedade de dispositivos e tamanhos de tela.

É necessário que os desenvolvedores compreendam os princípios fundamentais do design responsivo e saibam aplicar esses conceitos de forma eficaz para garantir que seus sites sejam acessíveis e utilizáveis por todos os usuários, independentemente do dispositivo que estão utilizando.

Dê o play!

Assimile

Neste infográfico, apresentamos resumidamente os principais conceitos relacionados a responsividade e adaptabilidade na web, dois tópicos fundamentais em aplicações web para a disciplina de desenvolvimento responsivo.



Referências

ALVES, William P. **Java para Web – Desenvolvimento de Aplicações**. São Paulo: Saraiva, 2015. [Minha Biblioteca]

ALVES, William P. **Projetos de Sistemas Web Conceitos, Estruturas**, Criação de Banco de dados e Ferramentas de Desenvolvimento. São Paulo: Saraiva, 2015. [Minha Biblioteca]

FERREIRA, Arthur G. **Interface de programação de aplicações (API) e web services**. São Paulo: Saraiva, 2021. [Minha Biblioteca]

MARCOLINO, Anderson da S. **Frameworks Front End**. São Paulo: Saraiva, 2021. [Minha Biblioteca]

STRASSACAPA, Renata M. *et al.* **Web Analytics**. Porto Alegre: Sagah, 2022. [Minha Biblioteca]

ZABOT, Diego; MATOS, Ecivaldo de S. **Aplicativos com bootstrap e angular – como desenvolver apps responsivos**. São Paulo: Saraiva, 2020. [Minha Biblioteca]

